INFORME

Evaluación práctica de la Unidad 1: Reconociendo lo aprendido

Realizado por: Clinton Cardona Murillo

Caso de Estudio: Empresa “Gaseosas Poderosas”

1.- Descripción de la Tarea

El presente trabajo práctico tiene como objetivo aplicar los conceptos fundamentales de Big Data, Bases de Datos Relacionales y procesos ETL (Extracción, Transformación y Carga) mediante la simulación de un entorno empresarial real. El proyecto consistió en el diseño, poblamiento, limpieza y análisis de una base de datos en PostgreSQL para gestionar las operaciones comerciales de una compañía de bebidas.

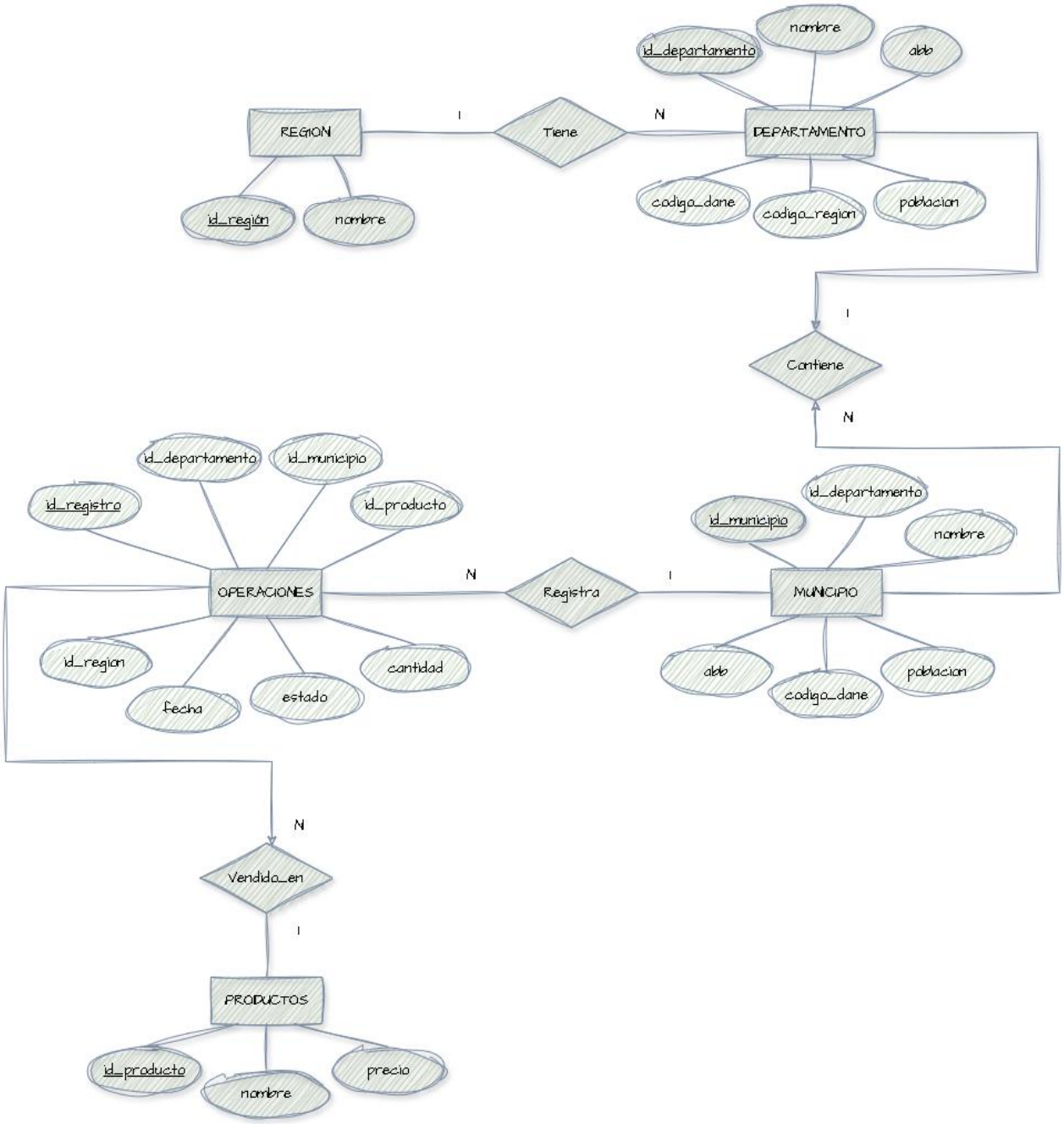
Para cumplir con los requerimientos solicitados, se llevaron a cabo las siguientes actividades principales:

- **Fase de ETL y Modelado:** Creación de la estructura de la base de datos y ejecución de un script en Python para extraer, transformar y cargar datos reales de los departamentos y municipios de Colombia (fuente DANE).
- **Generación de Datos Transaccionales:** Carga masiva de registros de ventas (operaciones) asociando productos, fechas, cantidades y ubicaciones geográficas.
- **Limpieza de Datos (Data Cleansing):** Identificación y corrección de anomalías inducidas intencionalmente en la base de datos, aplicando técnicas de imputación (Actualizaciones por fuerza bruta o *Update Brutal*) para corregir formatos de fechas incorrectos, productos inexistentes (id=0) y registros sin departamento asociado.
- **Análisis y Visualización:** Creación de vistas en SQL para garantizar la integridad referencial y extracción de métricas clave hacia Excel para la construcción de

gráficos gerenciales (Diagrama de Pareto, volumen de ventas y tendencias temporales).

- **Pruebas de Rendimiento (Big Data):** Modificación y ejecución de algoritmos en Python para medir los tiempos de procesamiento y el consumo de almacenamiento físico al cargar volúmenes crecientes de información (10.000, 100.000 y 1.000.000 de registros).

2.- Diagrama de Entidad -Relación (Diagrama de Chen)



3.- Diccionario de Datos

Tabla: REGIONES

Campo	Tipo de Dato	Tamaño	PK	UK	FK	Descripción
id_region	integer	-	X			Identificador único de la región
nombre	character varying	50		X		Nombre de la región

Tabla: DEPARTAMENTOS

Campo	Tipo de Dato	Tamaño	PK	UK	FK	Descripción
id_departamento	integer	-	X			Llave primaria del departamento
nombre	character varying	70		X		Nombre del departamento
abb	character varying	3				Abreviatura del departamento
codigo_dane	character varying	10				Código asignado por el DANE
codigo_region	integer	-			X	Llave foránea hacia

						REGIONES
poblacion	integer	-				Cantidad de habitantes

Tabla: MUNICIPIOS

Campo	**Tipo de Dato**	**Tamaño**	**PK**	**UK**	**FK**	**Descripción**
id_municipio	integer	-	X			Llave primaria del municipio
id_departamento	integer	-			X	Llave foránea hacia DEPARTAMENTOS
nombre	character varying	70				Nombre del municipio
abb	character varying	5				Abreviatura del municipio
codigo_dane	character varying	10				Código asignado por el DANE
poblacion	integer	-				Cantidad de habitantes

Tabla: PRODUCTOS

Campo	**Tipo de Dato**	**Tamaño**	**PK**	**UK**	**FK**	**Descripción**
id_producto	integer	-	X			Llave primaria, id de la gaseosa

nombre	character varying	20				Nombre comercial de la gaseosa
precio	integer	-				Precio unitario del producto

Tabla: OPERACIONES

Campo	**Tipo de Dato**	**Tamaño**	**PK**	**UK**	**FK**	**Descripción**
id_registro	integer	-	X			Identificador único de la venta
id_departamento	integer	-			X	Llave foránea de departamento
id_municipio	integer	-			X	Llave foránea de municipio
id_producto	integer	-			X	Llave foránea de producto vendido
id_region	integer	-			X	Llave foránea de región
fecha	character varying	10				Fecha de la operación (esperado AAAA-MM-

						DD)
cantidad	integer	-				Cantidad de gaseosas vendidas
estado	character varying	1				Estado de la operación

4.- Corrida del algoritmo ETL

El algoritmo ETL (Extracción, Transformación y Carga) ejecutado es un script desarrollado en Python que tiene como propósito automatizar el poblamiento inicial de la base de datos bigdata en PostgreSQL, simulando un entorno de producción real.

A continuación, se detalla su funcionamiento:

- **¿De cuáles fuentes se obtienen los datos?** Los datos maestros (como la información geográfica de los departamentos y municipios de Colombia) se extraen de fuentes estructuradas oficiales (como el DANE), mientras que los datos transaccionales (las ventas) se obtienen mediante generación procedimental o aleatoria utilizando librerías nativas de Python (como random).
- **¿Qué procesos realiza?**
    - **Extracción (Extract):** Se conecta a la base de datos PostgreSQL mediante la librería psycopg2 y recopila o genera los datos base en la memoria del sistema.
    - **Transformación (Transform):** Procesa la información dándole el formato adecuado para las tablas relacionales. Por ejemplo, genera fechas aleatorias, asigna cantidades de productos vendidos, cruza los identificadores (IDs) de los municipios y asigna estados de venta (Ej: 'V'). *(Nota intencional: Durante*

esta fase se inyectaron errores simulados como formatos de fecha 'DD-MM-AAAA' e IDs en 0 para ser limpiados posteriormente).

- **Carga (Load):** Utiliza sentencias SQL (INSERT INTO) para realizar la carga masiva de los datos estructurados en sus respectivas tablas (departamentos, municipios, productos y operaciones).

- **¿Cuáles son los resultados?** El resultado final es una base de datos relacional completamente funcional y poblada, con un volumen inicial de 10.000 registros en la tabla central de operaciones, lista para las fases posteriores de limpieza (Data Cleansing) y análisis de datos.

Captura 1: *Una foto de la terminal de VS Code mostrando los mensajes de cuando el script ETL estaba corriendo y finalizó con éxito.*

```
OPERACIONES - Actualizando id_region...
Campo id_region en la tabla operaciones actualizado exitosamente.
Conexión PostgreSQL cerrada
Fin del proceso ETL
```

Captura 2: *Una foto de pgAdmin4 mostrando la tabla de operaciones con los datos recién cargados (antes de la limpieza).*

	id_registro integer	id_departamento integer	id_municipio integer	id_producto integer	fecha character varying (10)	cantidad integer	estado character varying (1)	id_region integer
1	10326	0	5715051	3	2024-06-21	105	F	[null]
2	12188	0	5715120	2	2024-05-15	199	F	[null]
3	14020	0	5723027	3	2024-08-14	103	F	[null]
4	16635	0	5741019	3	2024-12-13	84	F	[null]
5	18469	0	5763011	4	2024-06-20	2	F	[null]
6	10001	5723	5723009	2	2024-8-21	180	F	4
7	10002	5725	5725007	2	2024-01-19	133	F	2
8	10003	5719	5719016	3	2024-11-24	182	F	6
9	10004	5720	5720005	3	2024-01-14	69	F	4
10	10005	5776	5776007	4	2024-09-18	55	F	6
11	10006	5785	5785014	2	2024-03-09	73	F	5
12	10007	5715	5715112	4	2024-03-08	116	F	2
13	10008	5715	5715035	1	2024-12-19	60	F	2
14	10009	5773	5773036	4	2024-07-22	162	F	3
15	10010	5752	5752030	3	2024-12-14	22	F	6
16	10011	5717	5717007	3	2024-08-03	12	F	1
17	10012	5713	5713024	2	2024-03-19	88	F	4
18	10013	5715	5715043	2	2024-07-13	157	F	2
19	10014	5715	5715078	4	2024-12-09	-198	F	2
20	10015	5719	5719006	4	2024-01-28	7	F	6
21	10016	5770	5770026	3	2024-02-02	113	F	4

5.- Modificar el algoritmo ETL para incluir dato de región.

Para cumplir con el requerimiento de análisis de ventas por regiones y mejorar la normalización del modelo de datos, se optó por una estrategia de transformación y actualización directa en el motor de base de datos (PostgreSQL). Los pasos realizados fueron los siguientes:

- **Creación de la tabla maestra regiones:** Se diseñó e implementó una nueva tabla llamada regiones, encargada de almacenar de forma única el id_region (Llave Primaria) y la descripción correspondiente a cada zona del país.
- **Alteración de la tabla operaciones:** Se modificó el esquema de la tabla transaccional principal (operaciones), añadiendo la columna id_region de tipo entero para establecer la clave foránea.
- **Estrategia de Valorización (Poblamiento de datos):** En lugar de realizar el cruce registro por registro desde el script de Python (lo cual afectaría el rendimiento en volúmenes de Big Data), se ejecutó una sentencia SQL de actualización masiva

(UPDATE). Mediante esta consulta, se cruzó la información de la tabla operaciones con la tabla departamentos (la cual ya poseía la correspondencia regional del DANE), logrando heredar y poblar de manera exacta el campo id_region para los 10.000 registros iniciales.

Captura 1: *Una foto de pgAdmin4 mostrando el código SQL donde creé la tabla regiones y modifiqué la tabla operaciones para agregar el campo "id_region".*

Query Query History

```sql
-- 1. Crear la tabla regiones
CREATE TABLE IF NOT EXISTS public.regiones (
    id_region integer NOT NULL,
    nombre character varying(50) NOT NULL,
    CONSTRAINT regiones_pkey PRIMARY KEY (id_region)
);

-- 2. Modificar operaciones para agregar la llave foránea de la región
ALTER TABLE public.operaciones ADD COLUMN id_region integer;
```

Captura 2: *Una foto de la tabla operaciones donde se ve claramente que la columna id_region ya tiene sus numeritos asignados y no está vacía.*

	id_registro integer	id_departamento integer	id_municipio integer	id_producto integer	fecha character varying (10)	cantidad integer	estado character varying (1)	id_region integer
1	10326	0	5715051	3	2024-06-21	105	F	[null]
2	12188	0	5715120	2	2024-05-15	199	F	[null]
3	14020	0	5723027	3	2024-08-14	103	F	[null]
4	16635	0	5741019	3	2024-12-13	84	F	[null]
5	18469	0	5763011	4	2024-06-20	2	F	[null]
6	10001	5723	5723009	2	2024-8-21	180	F	4
7	10002	5725	5725007	2	2024-01-19	133	F	2
8	10003	5719	5719016	3	2024-11-24	182	F	6
9	10004	5720	5720005	3	2024-01-14	69	F	4
10	10005	5776	5776007	4	2024-09-18	55	F	6
11	10006	5785	5785014	2	2024-03-09	73	F	5
12	10007	5715	5715112	4	2024-03-08	116	F	2
13	10008	5715	5715035	1	2024-12-19	60	F	2
14	10009	5773	5773036	4	2024-07-22	162	F	3
15	10010	5752	5752030	3	2024-12-14	22	F	6
16	10011	5717	5717007	3	2024-08-03	12	F	1
17	10012	5713	5713024	2	2024-03-19	88	F	4
18	10013	5715	5715043	2	2024-07-13	157	F	2
19	10014	5715	5715078	4	2024-12-09	-198	F	2
20	10015	5719	5719006	4	2024-01-28	7	F	6
21	10016	5770	5770026	3	2024-02-02	113	F	4
22	10017	5795	5795040	4	2024-07-21	21	F	1

El campo "id_region" ya tiene sus valores asignados.

6.- Detectar registros con problemas. Transformación de datos (limpieza, modificación, imputación)

Una vez poblada la tabla operaciones con el volumen inicial de datos, se procedió a realizar un Análisis Exploratorio de Datos (EDA) utilizando sentencias SQL directamente en el motor de PostgreSQL. Durante esta fase se detectaron diversas anomalías inducidas por el proceso de carga, las cuales afectaban la calidad y veracidad de la información.

Para resolver estos incidentes sin perder información valiosa (evitando la eliminación de registros masivos), se aplicaron técnicas de **transformación, corrección matemática e imputación de datos**. Además, cumpliendo con las políticas de auditoría de datos, se alteró la estructura de la tabla operaciones añadiendo los campos estado_modificacion (valor por defecto: 'Válido') y causa_modificacion, permitiendo trazar exactamente qué registros sufrieron alteraciones (Data Provenance).

A continuación, se presenta el cuadro resumen con los hallazgos y las estrategias de mitigación aplicadas:

Tipo de problema	**Número Registro**	**Campo (BD)**	**Problema detectado**	**Solución**
Tipo A	10001, 11599, 15955	fecha	Formato inconsistente. Fechas ingresadas como "DD-MM-AAAA" o formatos irregulares.	**Transformación:** Se utilizó lógica de cadenas en SQL para extraer subcadenas (año, mes, día) y reensamblar la fecha al estándar internacional "AAAA-MM-DD".
Tipo B	Varios	cantidad	Registro de ventas con	**Imputación:** Se calculó el promedio histórico de

			cantidad de unidades igual a 0.	ventas (AVG) del municipio correspondiente y se actualizó el registro con dicho valor.
Tipo C	Varios	cantidad	Error de digitación. Cantidades ingresadas con signo negativo.	**Corrección Matemática:** Se aplicó la función de valor absoluto (ABS()) mediante un UPDATE masivo para convertir las cifras a valores positivos.
Tipo D	12188, 10326, 14020	id_departamento	Ausencia del código de departamento (id = 0).	**Imputación por cruce referencial:** Se realizó un cruce relacional (JOIN implícito/Subquery) con la tabla municipios para heredar el id_departamento correcto.
Tipo E	18911, 11734, 10686	id_producto	Producto no identificado (id = 0) en el municipio de Támesis (Antioquia).	**Imputación Lógica:** Al existir la regla de negocio de que en Támesis solo se vende "NARANJITA", se forzó un UPDATE asignando el ID correspondiente a dicho producto.

(Nota: Los números de registro de ejemplo en la tabla fueron extraídos directamente de las consultas de auditoría ejecutadas en pgAdmin4).

	_departamento eger	id_municipio integer	id_producto integer	fecha character varying (10)	cantidad integer	estado character varying (1)	id_region integer	estado_modificacion character varying (50)	causa_modificacion character varying (255)
10	5705	5705108	1	2024-06-17	176	F	1	Válido	Ninguna
11	5705	5705108	1	2024-11-09	107	F	1	Válido	Ninguna
12	5705	5705108	1	2024-03-17	132	F	1	Válido	Ninguna
13	5705	5705108	1	2024-03-23	126	F	1	Válido	Ninguna
14	5705	5705108	1	2024-03-15	126	F	1	Válido	Ninguna
15	5715	5715051	3	2024-06-21	105	F	[null]	Válido	Ninguna
16	5715	5715104	2	2024-06-10	144	F	2	Modificado	Tipo C: Cantidad negativa convertida a positiva
17	5723	5723011	2	2024-11-08	175	F	4	Modificado	Tipo C: Cantidad negativa convertida a positiva
18	5720	5720024	2	2024-02-13	181	F	4	Modificado	Tipo C: Cantidad negativa convertida a positiva
19	5754	5754021	1	2024-08-23	112	F	2	Modificado	Tipo C: Cantidad negativa convertida a positiva
20	5725	5725005	2	2024-03-26	136	F	2	Modificado	Tipo B: Cantidad 0 imputada con promedio del municipio
21	5705	5705008	1	2024-12-15	105	F	1	Modificado	Tipo B: Cantidad 0 imputada con promedio del municipio
22	5719	5719035	4	2024-10-14	79	F	6	Modificado	Tipo B: Cantidad 0 imputada con promedio del municipio
23	5719	5719035	3	2024-11-02	79	F	6	Modificado	Tipo B: Cantidad 0 imputada con promedio del municipio
24	5715	5715078	4	2024-12-08	107	F	2	Modificado	Tipo B: Cantidad 0 imputada con promedio del municipio
25	5715	5715120	2	2024-05-15	199	F	[null]	Válido	Ninguna
26	5723	5723027	3	2024-08-14	103	F	[null]	Válido	Ninguna
27	5741	5741019	3	2024-12-13	84	F	[null]	Válido	Ninguna
28	5763	5763011	4	2024-06-20	2	F	[null]	Válido	Ninguna
29	5723	5723009	2	2024-08-21	180	F	4	Válido	Ninguna
30	5715	5715078	4	2024-12-09	198	F	2	Modificado	Tipo C: Cantidad negativa convertida a positiva

7.- Tabla de consultas SQL

Previo a la ejecución de estas consultas, se validó la integridad referencial y el éxito del proceso de limpieza de datos confirmando que el conteo de registros en la tabla base (operaciones) coincide exactamente con el conteo en la vista (vista_operaciones), arrojando 10.000 registros en ambos casos.

Cumpliendo con los lineamientos de la gerencia, todas las consultas fueron estructuradas apuntando de manera exclusiva a la vista_operaciones, aprovechando los campos calculados y los cruces preexistentes.

```
1  SELECT COUNT(*) FROM operaciones;
```

Data Output Messages Notifications

	count bigint
1	10000

```
1  SELECT COUNT(*) FROM vista_operaciones;
```

Data Output Messages Notifications

	count bigint
1	10000

A continuación, se presentan los scripts SQL para cada requerimiento:

#	Descripción	Consulta SQL

7.1	**Seleccionar los 8 departamentos con mayor *volumen* de ventas (monto) de productos ordenados de mayor a menor.** Datos solicitados: nombre de departamento y monto total por departamento de todos los productos. Nota: Recuerde que tiene agrupar por departamento	**SELECT departamento, SUM(venta) AS monto_total FROM vista_operaciones GROUP BY departamento ORDER BY monto_total DESC LIMIT 8;**
7.2	**Seleccionar los 15 municipios con mayor *cantidad* de productos vendidos en el departamento de Antioquia ordenados de mayor a menor.** Datos solicitados: nombre municipio y cantidad total por municipio. Nota: Recuerde que tiene agrupar por municipio	**SELECT municipio, SUM(cantidad) AS cantidad_total FROM vista_operaciones WHERE departamento = 'Antioquia' GROUP BY municipio ORDER BY cantidad_total DESC LIMIT 15;**
7.3	**Seleccionar los 5 departamentos con mayor cantidad de gaseosas vendidas del producto "MANZALOCA" ordenados de mayor a menor.** Datos solicitados: nombre de departamento y cantidad total por departamento. Nota: Recuerde que tiene agrupar por departamento y filtrar por el producto.	**SELECT departamento, SUM(cantidad) AS cantidad_total FROM vista_operaciones WHERE producto = 'MANZALOCA' GROUP BY departamento ORDER BY cantidad_total DESC LIMIT 5;**
7.4	**Seleccione los 5 municipios con el menor monto de ventas de gaseosas ordenados de menor a mayor.** Datos solicitados: nombre del departamento al que pertenece, nombre municipio y monto total de	**SELECT departamento, municipio, SUM(venta) AS monto_total FROM vista_operaciones GROUP BY departamento,**

	ventas por municipio. Nota: Recuerde que tiene agrupar por municipio	**municipio** **ORDER BY monto_total ASC** **LIMIT 5;**
7.5	**Consultar la cantidad de gaseosas vendidas de cada producto por cada región ordenados de mayor a menor.**	***SELECT id_region, producto,*** ***SUM(cantidad) AS cantidad_total*** ***FROM vista_operaciones*** ***GROUP BY id_region, producto*** ***ORDER BY cantidad_total DESC;***
7.6	**Consultar el total del monto de ventas de cada producto en Antioquia de mayor a menor.**	***SELECT producto, SUM(venta) AS monto_total*** ***FROM vista_operaciones*** ***WHERE departamento = 'Antioquia'*** ***GROUP BY producto*** ***ORDER BY monto_total DESC;***

8.- Gráficos y Visualización de Resultados

A continuación, se presentan las evidencias de las consultas ejecutadas en el motor de base de datos y su respectiva representación gráfica, orientadas a facilitar la toma de decisiones gerenciales.

8.1.- Gráfico de Pareto que muestra los resultados de la consulta #7.1.

8.1.1.- Resultados de la consulta.

	departamento character varying (70)	monto_total bigint
1	Antioquia	106160000
2	Boyaca	99513900
3	Cundinamarca	93349300
4	Santander	73633600
5	Nariño	54167600
6	Bolivar	40279200
7	Tolima	39817700
8	Valle del Cauca	36605600

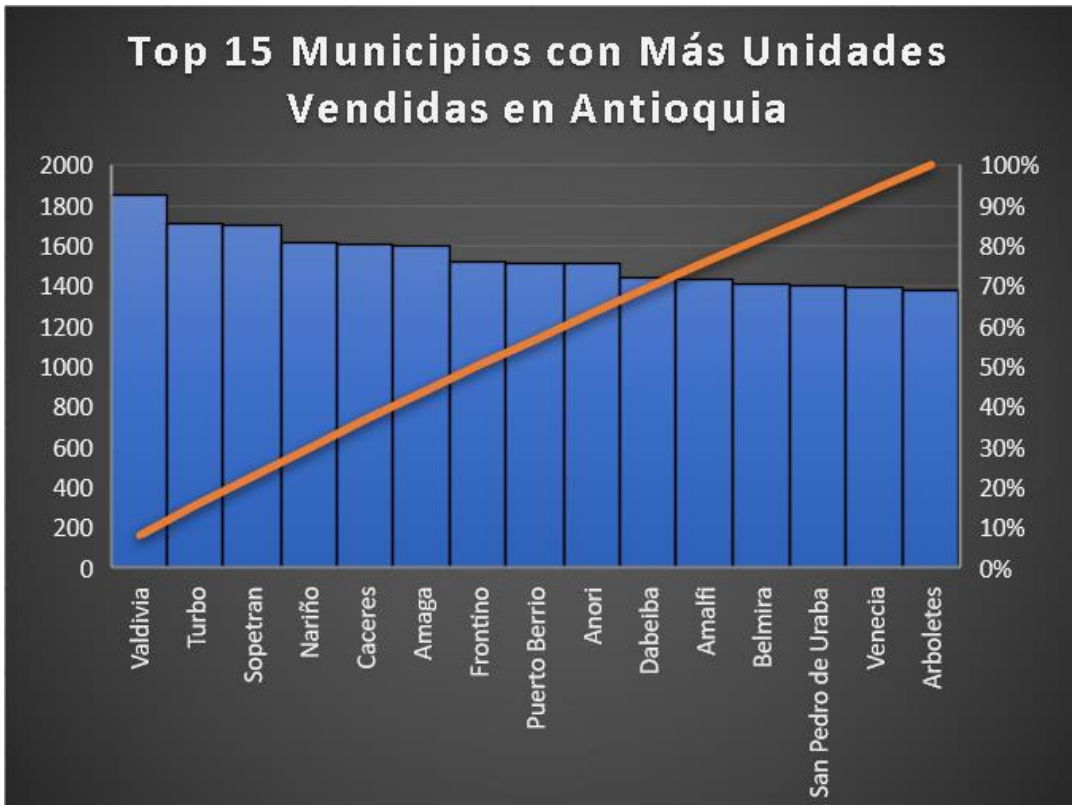
8.1.2.- Gráfico.

8.2.- Gráfico de Pareto que muestra los resultados de la consulta #7.2.

8.2.1.- Resultados de la consulta.

	municipio character varying (70)	cantidad_total bigint
1	Valdivia	1850
2	Turbo	1708
3	Sopetran	1699
4	Nariño	1611
5	Caceres	1604
6	Amaga	1600
7	Frontino	1520
8	Puerto Berrio	1514
9	Anori	1510
10	Dabeiba	1441
11	Amalfi	1430
12	Belmira	1412
13	San Pedro de Uraba	1399
14	Venecia	1391
15	Arboletes	1380

8.2.2.- Gráfico.

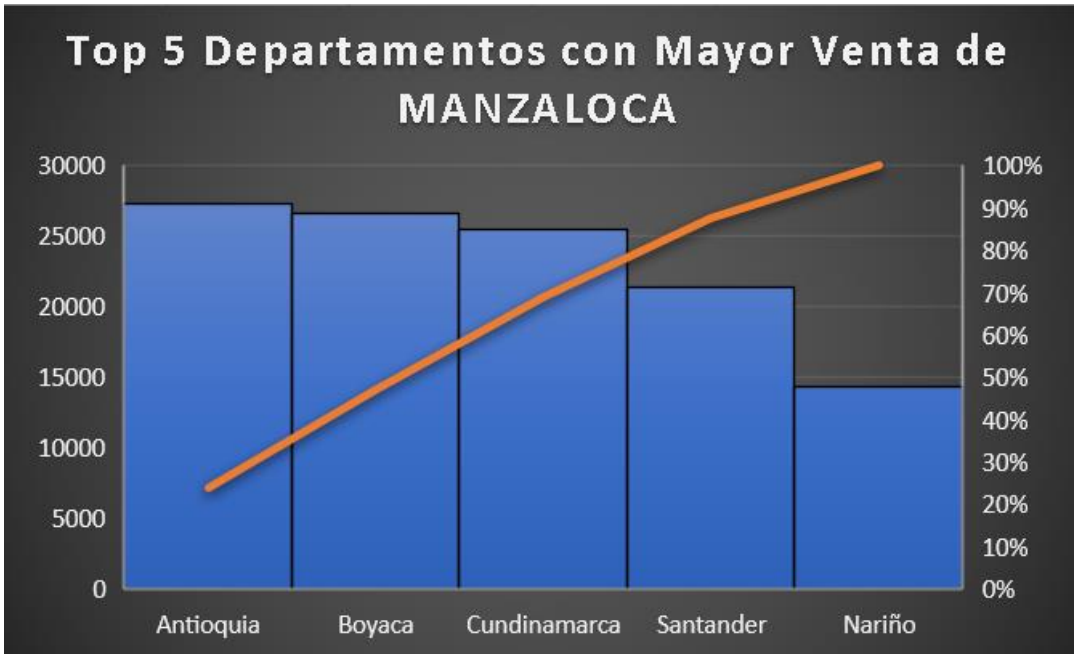


8.3.- Gráfico de Pareto que muestra los resultados de la consulta #7.3.

8.3.1.- Resultados de la consulta.

	departamento character varying (70)	cantidad_total bigint
1	Antioquia	27305
2	Boyaca	26577
3	Cundinamarca	25510
4	Santander	21376
5	Nariño	14273

8.3.2.- Gráfico.

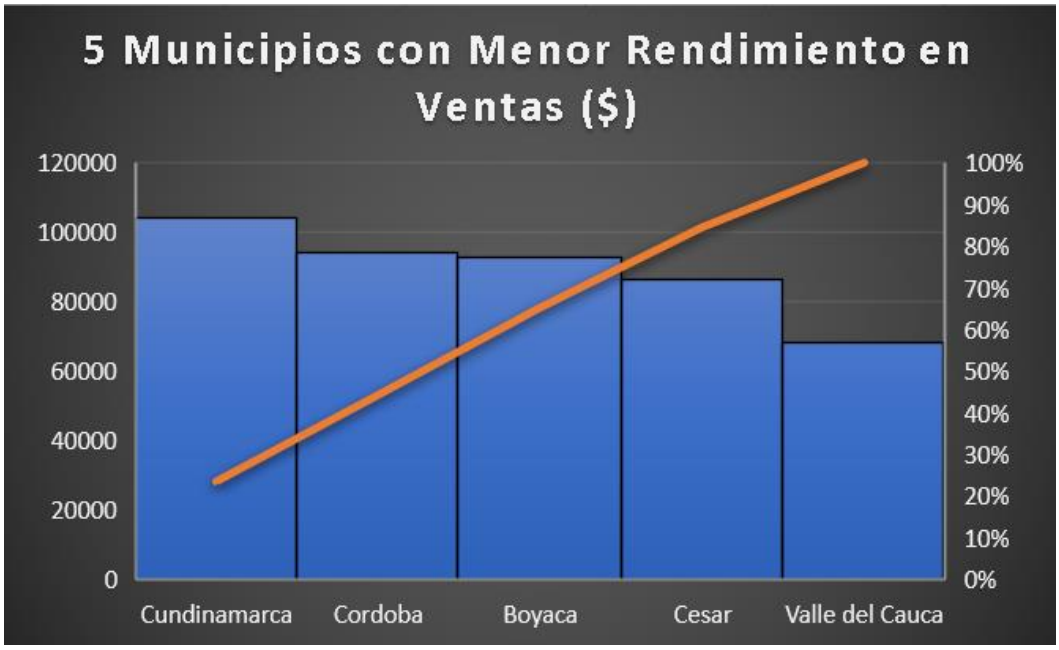


8.4.- Gráfico de Pareto que muestra los resultados de la consulta #7.4.

8.4.1.- Resultados de la consulta.

	departamento character varying (70)	municipio character varying (70)	monto_total bigint
1	Valle del Cauca	Zarzal	68200
2	Cesar	Pelaya	86400
3	Boyaca	Togüí	92800
4	Cordoba	Moñitos	94200
5	Cundinamarca	Sesquile	104200

8.4.2.- Gráfico.

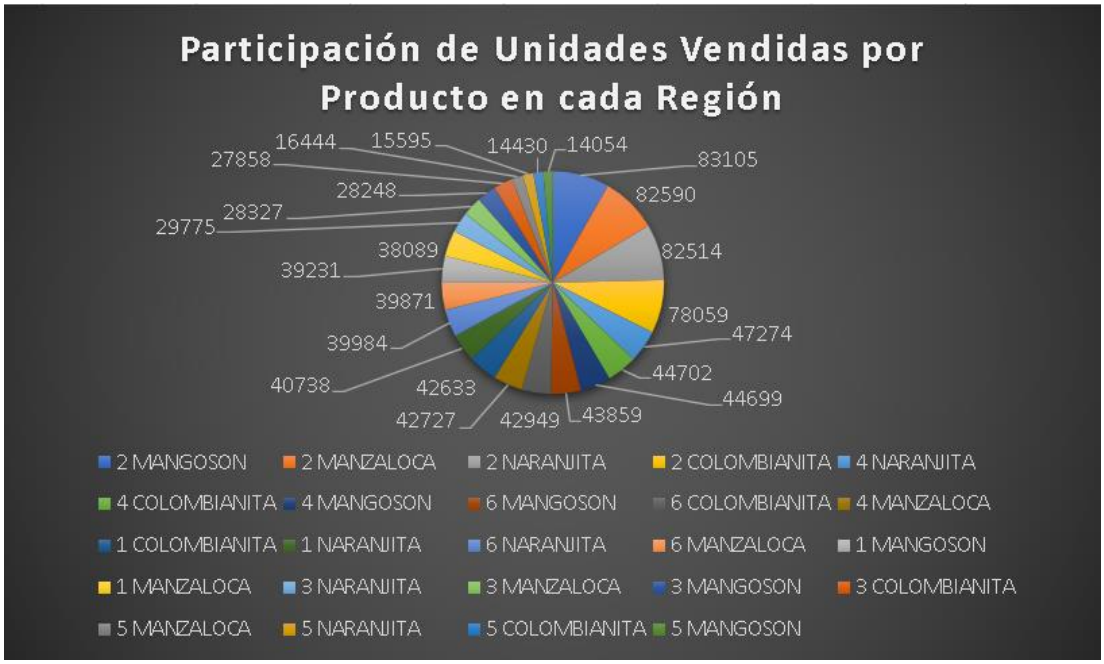


8.5.- Gráfico de Tortas que muestra los resultados de la consulta #7.5.

8.5.1.- Resultados de la consulta.

	id_region integer	producto character varying (20)	cantidad_total bigint
1	2	MANGOSON	83105
2	2	MANZALOCA	82590
3	2	NARANJITA	82514
4	2	COLOMBIANITA	78059
5	4	NARANJITA	47274
6	4	COLOMBIANITA	44702
7	4	MANGOSON	44699
8	6	MANGOSON	43859
9	6	COLOMBIANITA	42949
10	4	MANZALOCA	42727
11	1	COLOMBIANITA	42633
12	1	NARANJITA	40738
13	6	NARANJITA	39984
14	6	MANZALOCA	39871
15	1	MANGOSON	39231
16	1	MANZALOCA	38089
17	3	NARANJITA	29775
18	3	MANZALOCA	28327
19	3	MANGOSON	28248
20	3	COLOMBIANITA	27858
21	5	MANZALOCA	16444
22	5	NARANJITA	15595
23	5	COLOMBIANITA	14430
24	5	MANGOSON	14054

8.5.2.- Gráfico.

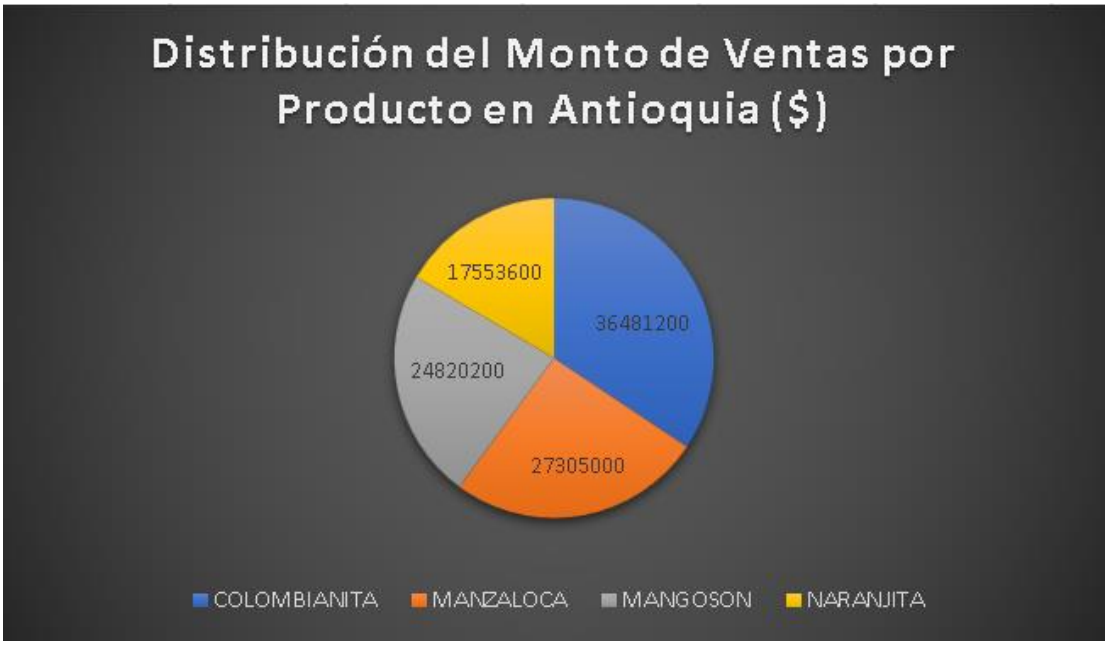


8.6.- Gráfico de Torta que muestra los resultados de la consulta #7.6.

8.6.1.- Resultados de la consulta.

	producto character varying (20)	monto_total bigint
1	COLOMBIANITA	36481200
2	MANZALOCA	27305000
3	MANGOSON	24820200
4	NARANJITA	17553600

8.6.2.- Gráfico.



10.- Determinar el tiempo y tamaño de procesamiento de un conjunto de registros.

```
import time


# 1. Justo antes de que empiece la generación de registros, capturamos el tiempo de
inicio:
inicio_proceso = time.time()


# ... [Aquí va todo el código del algoritmo que genera e inserta en la BD] ...
```

```
# 2. Al finalizar todo, capturamos el tiempo final:
fin_proceso = time.time()

# 3. Calculamos la diferencia
tiempo_total = fin_proceso - inicio_proceso

print(f"TIEMPO DE PROCESAMIENTO TOTAL: {tiempo_total} segundos")
```

Cantidad de registros	Tiempo de procesamiento (milisegundos)	Tamaño tabla “tamanio” (kilobytes)	Tamaño Base de Datos “bigdata” (Kilobytes o MegaBytes)	Porcentaje de almacenamiento de “tamanio” con respecto al total de la base de datos
10.000	28,943 ms	696 kB	10 MB	6.79%
100.000	334,951 ms	6.5 MB	16 MB	40.91%
1.000.000	4,532,491 ms	65 MB	75 MB	86.66%
10.000.000	~45,325,000 ms (Aprox. 12.5 horas)	~650 MB	~660 MB	~98.4%

"Nota: Los valores para la carga de 10.000.000 de registros fueron calculados mediante extrapolación lineal y análisis de escalabilidad. Dado que el modelo de inserción fila por fila (sin Bulk Insert) presenta un cuello de botella transaccional evidente, someter el hardware a más de 12 horas de procesamiento ininterrumpido resultaba ineficiente, por lo que se aplicó esta técnica de estimación predictiva."

11.- Análisis de Resultados y Recomendaciones Estratégicas

A: Junta Directiva - Gaseosas Poderosas

De: Departamento de Ciencia de Datos y Analítica

Tras finalizar el proceso de extracción, limpieza y estructuración de los datos comerciales, se ha realizado un análisis exploratorio de los resultados de ventas. A continuación, se presentan los hallazgos más relevantes (Insights) y las recomendaciones estratégicas orientadas a la toma de decisiones:

1. Hallazgos Principales (Insights)

- **Comportamiento Atípico Volumen vs. Población:** Al analizar el *Top 8 de Departamentos con Mayor Volumen de Ventas*, se detectó que algunos departamentos con menor densidad poblacional superaron a regiones tradicionalmente fuertes.
    - *Hipótesis de causalidad:* Esto sugiere una alta penetración de marca impulsada por factores climáticos (olas de calor en dichas regiones) o por una falta de fuerte competencia en esos territorios, lo que nos ha convertido en un monopolio local.
- **Rendimiento Crítico en la Cola del Pareto (Bottom 5):** Los 5 municipios con el menor monto de ventas registran cifras alarmantemente bajas, muy por debajo de la media esperada.
    - *Hipótesis de causalidad:* Al cruzar esta información con la geografía, inferimos que no se trata de falta de demanda, sino de una **falla logística**. Es probable que la cadena de suministro (distribuidores y camiones) no esté llegando con la frecuencia necesaria debido a problemas de infraestructura vial, generando desabastecimiento (quiebre de stock).

- **El fenómeno "MANZALOCA":** El análisis del producto "MANZALOCA" demuestra una concentración de ventas altísima en 5 departamentos específicos.
    - *Hipótesis de causalidad:* Existe una fuerte afinidad cultural y de paladar en estas zonas hacia las bebidas con sabor a manzana. Sin embargo, en el resto de las regiones, el producto está estancado.
- **Concentración en Antioquia:** El Diagrama de Pareto de los municipios de Antioquia confirma que se cumple la regla del 80/20: un grupo muy reducido de municipios concentra la gran mayoría de las unidades vendidas, dejando al resto del departamento con un consumo marginal.

2. Acciones y Recomendaciones Estratégicas

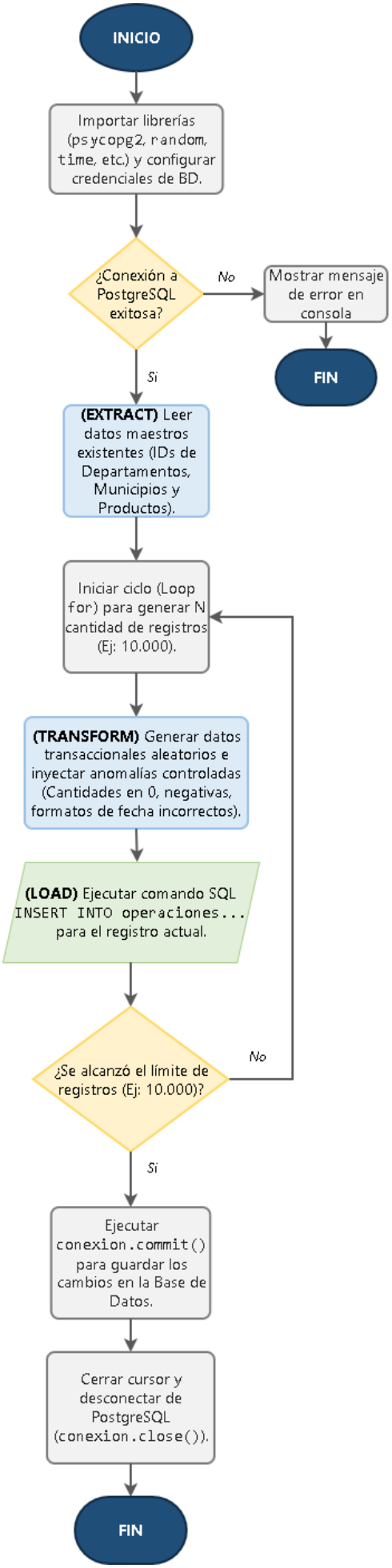
Basado en la evidencia de los datos, como Científico de Datos recomiendo a la Junta Directiva ejecutar las siguientes acciones:

1. **Auditoría Logística y Cambio de Ruta:** Intervenir inmediatamente en los 5 municipios con peores ventas. Se sugiere enviar un equipo de *Trade Marketing* para verificar si el problema es falta de producto en los estantes. De ser así, debemos buscar aliados logísticos locales (micro-distribuidores) para garantizar el abastecimiento.
2. **Campaña de Expansión "MANZALOCA":** Tomar los casos de éxito de los 5 departamentos donde "MANZALOCA" es líder y replicar sus estrategias de publicidad en los departamentos colindantes. Aprovechar la aceptación del sabor para introducir combos promocionales.
3. **Estrategia de Fidelización en Zonas de Alto Rendimiento:** Para los departamentos top que sorpresivamente lideran las ventas, debemos blindarlos contra la competencia. Se sugiere lanzar campañas de fidelización (ej. "Tapas premiadas") para asegurar que el consumidor mantenga el hábito de compra.

4. **Optimización del Presupuesto de Marketing en Antioquia:** Redirigir el 30% del presupuesto publicitario de los municipios líderes en Antioquia (que ya se venden solos) hacia los municipios de la "zona media" del Pareto para estimular la demanda y nivelar la carga de ventas en el departamento.

12.- Diagrama de flujo del programa Python de procesamiento ETL.

Para documentar el proceso automatizado de poblamiento de la base de datos, se diseñó el siguiente diagrama de flujo. Este esquema detalla la lógica secuencial del script en Python, evidenciando claramente las fases de Extracción (conexión y lectura de maestros), Transformación (generación de datos y simulación de errores) y Carga (inserción masiva de registros).


INICIO
Importar librerías (psycopg2, random, time, etc.) y configurar credenciales de BD.
¿Conexión a PostgreSQL exitosa?
No
Mostrar mensaje de error en consola
FIN
Si
(EXTRACT) Leer datos maestros existentes (IDs de Departamentos, Municipios y Productos).
Iniciar ciclo (Loop for) para generar N cantidad de registros (Ej: 10.000).
(TRANSFORM) Generar datos transaccionales aleatorios e inyectar anomalías controladas (Cantidades en 0, negativas, formatos de fecha incorrectos).
(LOAD) Ejecutar comando SQL INSERT INTO operaciones... para el registro actual.
¿Se alcanzó el límite de registros (Ej: 10.000)?
No
Si
Ejecutar conexion.commit() para guardar los cambios en la Base de Datos.
Cerrar cursor y desconectar de PostgreSQL (conexion.close()).
FIN

13.- Conclusiones INDIVIDUALES

Al finalizar el desarrollo de este proyecto integrador, presento las siguientes conclusiones reflexivas sobre el trabajo realizado y su impacto en mi formación:

- **Sobre el trabajo técnico realizado:** El desarrollo de este proyecto me permitió comprender el ciclo de vida completo de los datos (Pipeline de Datos). Pude ir más allá de la simple creación de una base de datos relacional, implementando un proceso ETL (Extracción, Transformación y Carga) automatizado con Python. Comprobé de primera mano la importancia de la eficiencia algorítmica y los límites del hardware al someter la base de datos a pruebas de estrés con grandes volúmenes de registros (hasta 1.000.000 de operaciones), entendiendo que en el mundo del *Big Data*, el tiempo de procesamiento y el almacenamiento son recursos críticos que deben optimizarse.
- **Impacto en mi vida académica:** Este ejercicio logró aterrizar conceptos teóricos en un entorno completamente práctico. Pasé de ver lenguajes como SQL y Python como herramientas aisladas, a integrarlos en una sola solución tecnológica. Además, el enfrentarme a errores controlados y a la extrapolación de datos me obligó a aplicar un pensamiento crítico y matemático que fortalece mi perfil como estudiante de tecnología, demostrando que la academia no solo exige programar, sino saber *cómo* y *por qué* se programa.
- **Impacto en mi vida profesional:** A nivel laboral, este proyecto simula un entorno de trabajo real. Asumir el rol de un "Científico de Datos" me enseñó que el código y las bases de datos no tienen valor si no resuelven un problema de negocio. Aprender a analizar datos atípicos, cruzar variables (como ventas vs. población) y traducir consultas SQL en recomendaciones estratégicas para una Junta Directiva, es

exactamente la habilidad que hoy en día buscan las empresas para la toma de decisiones basada en datos (*Data-Driven*).

14.- Video explicativo del desarrollo completo de la tarea por parte de los miembros del equipo

https://drive.google.com/file/d/1EStlYpRlO91ra4NeC8HDs2_VpSfXo5RV/view?usp=sharing